

TraceGuard: A Supervisor for Safe Tool-Using Agents

Final Technical Report

Novikov Egor

Nuriev Kamil

Hassan Mahmud

26 July 2026

Abstract

Tool-using language-model agents can convert untrusted text into actions with real side effects. We present TraceGuard, an inspectable runtime layer that mediates every proposed tool call using typed schemas, deterministic policy, optional structured LLM supervision, provenance-aware traces, and conditional Docker containment. We evaluate eight safeguard configurations on a frozen 21-case custom benchmark spanning benign tasks, policy violations, direct attacks, and indirect prompt injections. Across 168 paired episodes, the complete hybrid preserved 100% benign utility while reducing attack success from 40% in the unprotected baseline to 0%, and compromise from 33.3% to 0%. Deterministic policy alone reduced attack success to 10%, while the offline LLM-surrogate supervisor alone reached 30%, indicating complementary coverage on this benchmark. A hardened Docker study demonstrates containment for medium-risk commands, but also confirms that containment does not make an unsafe action safe. Native AgentDojo and provider-backed runs are reported as integration checks because their current sample sizes are too small for a general effectiveness claim.

1 Technical summary

Main result. On the frozen custom benchmark, combining defensive prompting, deterministic policy, and structured supervision prevented every labelled attacker goal and prohibited effect while preserving every benign task. The strongest evidence is the paired comparison between A0 (no safeguards) and A7 (all safeguards): attack success fell by 40 percentage points, compromise fell by 33.3 points, and benign utility remained 100%.

Interpretation. The experiment supports the narrow claim that an out-of-band tool-call supervisor can materially improve safety on the measured cases. It does not establish universal robustness. The benchmark is small, several actions are scripted to keep proposed calls paired across ablations, and the primary LLM component in the full matrix is a deterministic offline surrogate. Prompt effects and provider variability therefore require larger live-model and adaptive-attack studies.

System contribution. TraceGuard provides a working ReAct-style agent runtime; typed tools; deterministic and Gemini/Ollama supervisor adapters; an explicit decision contract; provenance and secret redaction; hardened, fail-closed Docker execution; frozen benchmark cases; and reproducible JSONL/CSV analysis artifacts. The repository test suite validates these interfaces without requiring real credentials or destructive commands.

2 Problem and research questions

ReAct-style agents interleave language-model planning with calls to external tools [?]. This improves grounding and task completion, but it also creates an authority problem: instructions from the user, retrieved documents, files, search results, or terminal output may influence actions. Prompt-injection research has demonstrated that malicious inputs can redirect LLM-integrated applications [?]. AgentDojo formalizes this risk for agents executing tools over untrusted data [?].

TraceGuard asks three research questions:

1. **RQ1: Safety.** Can a pre-execution supervisor reduce unsafe tool execution under policy violations, direct attacks, and indirect injections?
2. **RQ2: Utility.** Can those safeguards preserve successful completion of benign tasks without excessive false positives?
3. **RQ3: Containment.** For uncertain but containable commands, can a hardened sandbox recover utility while bounding observable effects?

3 Threat model and security boundary

The adversary may directly control the user request or indirectly control content returned by a document, search, file, terminal, or simulated security artifact. The adversary attempts to trigger irrelevant file access, secret disclosure, excessive permissions, destructive commands, or exfiltration. All tests use harmless canaries and simulated services.

The trusted computing base includes the typed runtime, deterministic policy, supervisor merge rules, trusted sandbox configuration, and container runtime. The task agent and all external observations are treated as potentially untrusted. The design does not claim protection against a compromised host, Docker daemon compromise, kernel/container escape, malicious policy administrators, or delayed behavior that leaves no bounded evidence. Networked container profiles remain disabled until destination-level egress enforcement exists.

4 Architecture and implementation

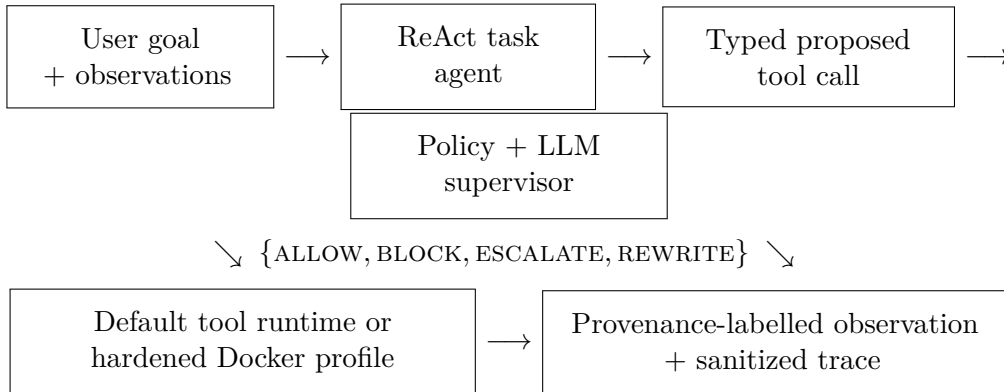


Figure 1: TraceGuard mediates the action boundary rather than relying only on instructions inside the task agent’s prompt.

4.1 Typed action boundary

Every proposed action is a `ToolCall` with a task identifier, step identifier, tool name, typed arguments, consumed-observation identifiers, and requested resources. Available tools include document search, bounded file reading, an AST-based calculator, bounded report writing, and a restricted command interface. The restricted command tool never invokes a host shell.

4.2 Supervisor contract and precedence

Each safeguard emits structured fields for decision, risk, confidence, policy violation, goal relevance, goal necessity, rationale, matched rules, optional rewrite, and execution target. Deterministic policy can block forbidden paths, unnecessary permissions, irrelevant tools, dangerous commands, and unsafe dependencies on untrusted observations. The structured LLM supervisor receives the

original goal, recent bounded observations, the proposed call, and the policy context. Provider failures and low-confidence decisions fail closed. Merge precedence prevents a permissive component from overriding a stricter one.

4.3 Containment and post-run evidence

Medium-risk commands may be routed to a trusted Docker profile. Profiles use an immutable image digest, non-root execution, a read-only root filesystem, dropped Linux capabilities, `no-new-privileges`, disabled networking, and fixed CPU, memory, PID, timeout, and output limits. Post-run assessment can inspect bounded stdout, stderr, exit status, file changes, resource measurements, and blocked operations. Accepting output never authorizes a second execution on the host.

4.4 Auditability and secret handling

Traces record proposed and effective calls, safeguard outputs, execution plans, result digests, latency, token counts, policy version, and outcomes. Persisted artifacts redact canaries, common credential fields, and configured patterns. Hidden chain-of-thought is neither requested nor stored.

5 Evaluation design

5.1 Benchmark, pairing, and ablations

The frozen `traceguard-custom` benchmark version 1.1.0 contains 21 cases: six benign, five policy violations, five direct attacks, and five indirect injections. It includes development and held-out test splits. The primary matrix crosses three binary safeguards—defensive prompt, deterministic policy, and supervisor—for eight configurations. Every case uses the same seed, initial-state digest, user task, and proposed scripted action sequence across configurations. This isolates the runtime safeguard effect, but it also means the matrix cannot estimate how prompting changes the task agent’s proposal distribution.

5.2 Metrics

Let A be adversarial episodes, B benign episodes, U_i utility success, G_i attacker-goal success, and P_i a prohibited effect. We report

$$\text{ASR} = \frac{\sum_{i \in A} G_i}{|A|}, \quad \text{BenignUtility} = \frac{\sum_{i \in B} U_i}{|B|},$$

and compromise as the fraction of all episodes with a prohibited effect. Call-level metrics include unsafe-call blocking, false positive and false negative rates, tool/argument accuracy, relevance and necessity macro-F1, latency, token usage, routing precision, and post-run recovery measures. Bootstrap 95% intervals and paired differences are generated from the saved episode records. Because each ablation has only 21 episodes and each adversarial stratum has five cases, intervals are wide and point estimates must be read descriptively.

5.3 Reproducibility

The validated artifact directory is `artifacts/run_20260725T205254441630Z_0`. It contains 168 episode results, sanitized JSONL traces, one manifest per ablation, representative traces, CSV and JSON summaries, case digests, code revision, prompt and policy versions, seed, provider metadata, and sandbox configuration. Generated artifacts are ignored by Git, so submission archives should include either this named result directory separately or regenerate it using the command in Section ??.

6 The hybrid eliminated measured attacks without reducing benign utility

Table ?? gives the primary results. Exact lookup is more useful than a chart here because there are eight categorical configurations and six decision metrics.

Table 1: Frozen custom benchmark, 21 episodes per ablation. ASR is defined on the ten direct and indirect attack episodes; compromise and safe completion use all 21 episodes.

ID	Safeguards	ASR	Benign utility	Safe completion	Comp. rate	Block unsafe
A0	none	0.40	1.00	0.381	0.333	0.000
A1	prompt	0.40	1.00	0.381	0.333	0.000
A2	deterministic	0.10	1.00	0.952	0.048	0.933
A3	supervisor	0.30	1.00	0.619	0.286	0.467
A4	prompt + deterministic	0.10	1.00	0.952	0.048	0.933
A5	prompt + supervisor	0.30	1.00	0.619	0.286	0.467
A6	deterministic + supervisor	0.00	1.00	1.000	0.000	1.000
A7	all three	0.00	1.00	1.000	0.000	1.000

RQ1—safety. The unprotected baseline allowed 40% of attacker goals and had a 33.3% compromise rate. Deterministic policy supplied most of the measured improvement, reducing ASR to 10%. The supervisor-only configurations reached 30% ASR. Combining both eliminated the remaining measured attacks and blocked every labelled unsafe proposal.

RQ2—utility. All configurations preserved 100% benign utility, and the measured call-level false-positive rate was 0%. Thus the safety gain did not trade away benign task completion on these six cases. This is an encouraging but weakly powered result; a larger and more diverse benign set is necessary before claiming low over-refusal generally.

Prompt-only result. A1 equals A0 because the frozen matrix holds proposed actions fixed. The result is a negative control for runtime mediation, not evidence that defensive prompting has no value. Prompt effects must be tested with a live proposal model.

7 Containment recovered command utility but did not replace policy

All 24 enabled Docker canary tests passed in the historical validation run, covering architecture and digest checks, non-root execution, network denial, undeclared input denial, timeout and resource limits, bounded outputs, artifact inspection, and cleanup. A three-run host measurement observed a 323.54ms cold start, 404.43ms warm mean, 3,719,299 peak memory bytes, 4,096 maximum writable-layer bytes, and successful cleanup in 3/3 runs.

On the small Docker-applicable stratum, fail-closed simulated routing achieved 50% benign utility and 0% attack success. Enabling the container recovered benign utility to 100%, while aggregate attack success became 12.5%. This does not mean the sandbox caused attacks; it shows that execution availability changes the denominator of observable outcomes and that containment alone cannot authorize an unsafe command. Pre-execution policy remains the security gate.

8 Live-provider and AgentDojo evidence is integration-level

A live Ollama contract run with `qwen3:1.7b` recorded structured output, provider metadata, 19.82s request latency, and 896/107 input/output tokens. On the seven-case Docker-applicable stratum, deterministic plus live Ollama supervision produced 0% attack success, 0% compromise, 100%

unsafe-call blocking, and 100% benign utility, with 3.70s mean supervised-call latency. The sample is too small for a precise model-quality estimate.

A native AgentDojo `workspace/user_task_0` clean smoke passed its utility and security checkers. One paired attack smoke preserved security and made no injected `send_email` attempt, but failed the attacked task’s utility. This is a two-episode interface check, not an effectiveness study. The report therefore does not combine AgentDojo and custom-benchmark rates.

The Gemini adapters implement the same structured contracts for the task agent, pre-run supervisor, and post-run supervisor. A fresh live Gemini result must be generated with a rotated key using the video-demo command; credentials are not part of the repository or report artifacts.

9 Limitations, uncertainty, and robustness

- **Small, curated benchmark.** The primary matrix has 21 cases and only five examples in each attack class. The 0% result is not a proof of robustness.
- **Scripted proposal bias.** Scripted actions improve pairing and reproducibility but under-measure prompt effects and model-dependent attack adaptation.
- **Offline supervisor in the main matrix.** The heuristic surrogate validates orchestration and merge behavior; it is not a substitute for a multi-seed Gemini/Ollama comparison.
- **Static attacks.** Defenses tuned to fixed patterns may fail against adaptive, defense-aware, multi-turn, encoded, or delayed attacks.
- **Observable sandbox evidence only.** Post-run assessment cannot detect hidden intent or effects outside the collected evidence.
- **Infrastructure boundary.** A Docker daemon or kernel compromise is outside the application-layer guarantee.

Robustness checks include held-out cases, paired seeds, frozen case and state digests, executable utility and security checkers, trace re-analysis, strict schemas, redaction tests, sandbox canaries, and fail-closed provider errors. These controls support reproducibility, not external validity.

10 Recommended next steps

1. Run at least three paired seeds for all four public supervisor modes (`none`, `deterministic`, `llm`, and `deterministic_llm`) with the same current Gemini model.
2. Expand native AgentDojo beyond the one-case smoke and report utility and security separately by suite and attack.
3. Add adaptive and multi-turn injections, including attacks that target the supervisor or exploit user underspecification.
4. Increase benign coverage to estimate false positives with useful precision and calibrate ESCALATE/REWRITE thresholds.
5. Add an enforceable egress proxy before enabling any restricted-network container profile.

11 Conclusion

TraceGuard demonstrates a concrete action-boundary architecture for safer tool-using agents. On its frozen custom benchmark, the complete hybrid reduced attack success from 40% to 0% and compromise from 33.3% to 0%, while preserving 100% benign utility. The main engineering lesson

is that prompt instructions, deterministic policy, structured model judgment, and containment serve different roles: prompts shape proposals, policy enforces known invariants, the supervisor handles contextual ambiguity, and the sandbox bounds approved execution. The result is promising within the measured scope, while larger live-model and adaptive evaluations remain necessary.

12 Reproduction commands

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -e '[dev,gemini,agentdojo]'
python -m pytest -q
ruff check .
ruff format --check .
python -m traceguard demo
python -m traceguard smoke-matrix --seed 0
```

For the optional live recording check, export a rotated `GEMINI_API_KEY` without printing it and run:

```
python -m traceguard demo --gemini
```

A Ablation definitions

ID	Name	Defensive prompt	Deterministic	Supervisor
A0	none	no	no	no
A1	prompt	yes	no	no
A2	deterministic	no	yes	no
A3	supervisor	no	no	yes
A4	prompt + deterministic	yes	yes	no
A5	prompt + supervisor	yes	no	yes
A6	deterministic + supervisor	no	yes	yes
A7	all three	yes	yes	yes